

HomeService OS

Complete Project Planning Document

Project: AI-ready Home Maintenance, Moving & Local Service Platform

Primary Stack: HTML, CSS, JavaScript, Python Flask, MySQL, MySQL Workbench

AI: Planned for a later phase, after the core product is stable

1. Project Objective

- A platform for managing home services, appliances, maintenance, technicians, bookings, expenses and relocation/reinstallation.
- The system maintains a digital history of the home and its appliances and can later support an AI agent.

2. User Roles

- **Customer:** Register/login, manage homes and appliances, request services, find nearby technicians, book services, track status, view service history, expenses, reminders, ratings and Move Mode.
- **Technician:** Register, create profile, add skills, set location/availability, receive and accept requests, update jobs, record work and charges, view earnings and ratings.
- **Admin:** Manage customers, technicians, services, bookings, complaints, payments and platform analytics.

3. Core Product Flow

- Customer → Home → Appliances → Maintenance → Service Request → Technician → Booking → Service → Payment → Service History.
- Move flow: Old Home → New Home → Transfer Appliances → Identify Required Services → Find Nearby Technicians → Book → Complete Installation.

4. Home Management

- Support multiple homes with address, city, state, pincode, latitude, longitude, home type and current/previous status.

5. Appliance Management

- Store appliance name, category, brand, model, purchase date, warranty date and associated home.
- Categories can include fans, lights, AC, refrigerator, washing machine, TV, geyser, RO purifier, microwave, inverter and other appliances.

6. Service Management

- Initial services: Electrical, Plumbing, AC Service, Appliance Repair, Carpentry, Painting, Cleaning, RO Service, Geyser Service and Installation.
- Each service can store name, description, estimated price/range, required skill and estimated duration.

7. Service Request & Booking

- Customer selects service → appliance → problem → preferred date/time → location → submit request.
- Statuses: Pending, Technician Assigned, Accepted, Scheduled, In Progress, Completed, Cancelled.

8. Nearby Technician Matching

- Find technicians using customer location, required skill, availability, rating, experience and estimated price.
- Start with city/pincode matching if needed, then add latitude/longitude distance calculation.

9. Move Mode – Major Feature

- Customer selects 'Move to New Home', enters the new location and chooses appliances to transfer.
- System identifies required removal/installation services, finds technicians near the new home, estimates cost and allows individual or complete move-setup booking.
- Example: 4 fans → fan installation; 2 ACs → AC removal/installation; washing machine → appliance installation; RO → purifier installation.

10. Cost Estimation

- Show an estimated range before booking. Example: Fan installation ×4 ■800; Light installation ×6 ■600; AC installation ×2 ■3,000; Washing machine ■500; RO ■400; estimated total ■5,300.

11. Service History & Maintenance

- Maintain appliance-specific service history with date, service type and cost.
- Use simple rule-based maintenance reminders initially, such as an AC service reminder after the configured service interval.

12. Ratings, Reviews & Payments

- After completion, customers can rate service quality, punctuality and professionalism.
- Initial payment module can store booking ID, service cost, additional cost, total, payment method, payment status and payment date without implementing a payment gateway.

13. Statistics & Analytics

- Customer: total services, spending, average service cost, most serviced appliance, monthly expenses.
- Technician: jobs completed, average rating, service time, cancellation rate, earnings and repeat customers.
- Admin: total customers, technicians, bookings, completed/cancelled services, revenue, average booking value, popular services and location-wise demand.
- Python can be used for mean, median, mode, variance, standard deviation, quartiles, correlation, regression and appropriate hypothesis tests.

14. MySQL Database

- Initial tables: users, customers, technicians, homes, appliances, services, technician_skills, technician_availability, service_requests, bookings, service_history, payments, reviews, complaints, notifications, maintenance_reminders, move_requests, move_appliances.

15. Suggested Project Structure

- app.py; config.py; requirements.txt; templates/; static/css/; static/js/; routes/auth.py; routes/customer.py; routes/technician.py; routes/admin.py; routes/services.py; routes/booking.py; routes/move.py; services/technician_matching.py; services/maintenance.py; services/statistics.py; database/connection.py; database/queries.py.

16. Development Phases

- **Phase 1 – Planning:** Requirements, roles, use cases, ER diagram, database design, UI wireframes.
- **Phase 2 – UI:** Landing page, login, registration and dashboards.
- **Phase 3 – Backend:** Flask, MySQL connection, CRUD, authentication and role management.
- **Phase 4 – Core Product:** Homes, appliances, services, requests, technicians, bookings, service history and reviews.
- **Phase 5 – Nearby Matching:** Location, distance, skill and availability matching.
- **Phase 6 – Move Mode:** New home, appliance transfer, required services, technician search, cost estimation and multi-service booking.
- **Phase 7 – Statistics:** Analytics, calculations, charts and reports.
- **Phase 8 – Testing:** Functional, database, security and workflow testing.
- **Phase 9 – Deployment:** Production backend, database and website deployment.

17. AI Agent – Later Phase

- Do not build the AI agent in the first version.
- Later, connect an AI agent to controlled backend functions such as `get_customer()`, `get_home()`, `get_appliances()`, `get_service_history()`, `check_booking()`, `find_nearby_technicians()`, `check_availability()`, `create_service_request()` and `create_booking()`.
- Example future request: 'I'm moving next week and need my AC and fans installed at my new house.' The agent can understand the request, identify appliances, create a move request, find required services, find technicians, ask for confirmation and create bookings.

18. Recommended MVP

- 1. Login and roles
- 2. Home management
- 3. Appliance management
- 4. Service requests
- 5. Technician management
- 6. Nearby technician matching
- 7. Booking and tracking
- 8. Move Mode
- 9. Service history
- 10. Statistics dashboard
- After these work properly, add the AI agent.

Final Architecture

Layer	Technology / Responsibility
Frontend	HTML + CSS + JavaScript
Backend	Python + Flask

Database	MySQL
Database Management	MySQL Workbench
Statistics	Python + Pandas / NumPy / SciPy
Charts	JavaScript charting or simple HTML/CSS initially
Future AI	LLM + controlled backend tools

Key development principle: Build the core product first. Make the backend modular so the AI agent can be added later without rebuilding the application.